

ITA0609
MACHINE LEARNING
CO2 – AT2
Submitted By,
T.Gobika(192312286)

Question 1

A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.

Introduction

K-Means is an unsupervised machine learning algorithm used to group similar data points into clusters. Sometimes, the algorithm produces different cluster assignments even for similar datasets due to initialization issues, incorrect parameter selection, or the presence of outliers.

Possible Reasons for Inconsistent Clustering

1. Poor Initialization

- K-Means randomly selects initial centroids.
- Different initial centroids may lead to different clustering results.
- The algorithm may converge to a local optimum instead of the global optimum.

2. Incorrect Number of Clusters (K)

- Choosing too few clusters merges different groups.
- Choosing too many clusters unnecessarily splits natural groups.
- Incorrect K decreases clustering accuracy.

3. Sensitivity to Outliers

- Outliers pull centroids away from actual cluster centers.
- This leads to incorrect cluster formation.

4. Unscaled Features

- Features with larger numerical values dominate distance calculations.
- Euclidean distance becomes biased.

Debugging Approaches

Check Cluster Initialization

- Run the algorithm multiple times.
- Compare clustering results.

Visualize the Data

- Plot the dataset to inspect natural clusters.
- Identify outliers.

Evaluate Cluster Quality

- Calculate the Silhouette Score.
- Higher silhouette values indicate better clustering.

Inspect Feature Scaling

- Verify whether all features are normalized.

Optimization Strategies

K-Means++ Initialization

- Selects better initial centroids.
- Produces faster convergence.
- Improves clustering consistency.

Normalization

- Scale features using StandardScaler or MinMaxScaler.
- Prevents dominance of large-valued features.

Silhouette Analysis

- Helps determine the optimal number of clusters.
- Maximizes separation between clusters.

Remove Outliers

- Detect outliers using statistical methods.
- Improves centroid placement.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Load Dataset
data = load_iris()
X = data.data

# Normalize Data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# K-Means with K-Means++
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
labels = kmeans.fit_predict(X_scaled)

# Silhouette Score
score = silhouette_score(X_scaled, labels)

print("Cluster Labels:")
print(labels)

print("Silhouette Score:", score)
```

Conclusion

K-Means clustering performance depends on proper centroid initialization, selecting the correct number of clusters, handling outliers, and feature scaling. Using K-Means++, normalization, and silhouette analysis significantly improves clustering quality.

Question 2

A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

Introduction

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression. It performs well on linear and nonlinear data when appropriate kernels and parameters are selected.

Possible Reasons for Poor Performance

1. Incorrect Kernel Selection

- Linear kernel cannot capture nonlinear relationships.
- Choosing the wrong kernel reduces accuracy.

2. Improper Hyperparameter Tuning

- Incorrect values of C and Gamma lead to overfitting or underfitting.
- Model performance decreases.

3. Feature Scaling Problems

- SVM depends on distance calculations.
- Features with different scales affect the decision boundary.

4. Noisy Data

- Incorrect labels reduce model accuracy.

Debugging Techniques

Test Different Kernels

- Linear
- Polynomial
- Radial Basis Function (RBF)
- Sigmoid

Perform Cross Validation

- Evaluate model performance using different folds.

Analyze Confusion Matrix

- Identify classification errors.

Check Data Scaling

- Ensure all features are standardized.

Optimization Strategies

Kernel Selection

- Use RBF kernel for nonlinear datasets.

Hyperparameter Tuning

- Optimize C and Gamma using GridSearchCV.

Feature Scaling

- Apply StandardScaler before training.

Cross Validation

- Prevents overfitting.
- Selects the best parameters.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score

# Load Dataset
data = load_iris()
X = data.data
y = data.target

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Hyperparameter Tuning
parameters = {
    'C': [0.1, 1, 10],
    'gamma': [1, 0.1, 0.01],
    'kernel': ['rbf', 'linear']
}

grid = GridSearchCV(SVC(), parameters, cv=5)
grid.fit(X_train, y_train)

# Prediction
prediction = grid.predict(X_test)

print("Best Parameters:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, prediction))
```

Conclusion

SVM performance can be improved by selecting the appropriate kernel, tuning hyperparameters, and scaling features. GridSearchCV and StandardScaler help achieve better classification accuracy.

Question 3

A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies.

Introduction

Recurrent Neural Networks (RNNs) are widely used for Natural Language Processing tasks such as text classification, language translation, and sentiment analysis. Standard RNNs often fail to learn long-term dependencies because of vanishing gradients and limited memory.

Causes of Poor Long-Term Dependency Learning

1. Vanishing Gradient Problem

- Gradients become very small during backpropagation.
- Earlier information is forgotten.

2. Exploding Gradients

- Very large gradients make training unstable.

3. Limited Memory

- Standard RNNs remember only recent information.
- Long sentences become difficult to process.

4. Insufficient Training

- Poor learning rate or fewer epochs reduce performance.

Debugging Steps

Monitor Training Loss

- Observe whether loss decreases properly.

Visualize Gradient Values

- Detect vanishing or exploding gradients.

Evaluate Validation Accuracy

- Compare training and validation performance.

Experiment with Different Models

- Compare RNN, LSTM, and GRU.

Optimization Strategies

Use LSTM

- Stores long-term information using memory cells.
- Solves vanishing gradient problems.

Use GRU

- Simpler than LSTM.
- Faster training with similar performance.

Attention Mechanism

- Focuses on important words.
- Improves context understanding.

Gradient Clipping

- Prevents exploding gradients.

Better Training Strategies

- Use Adam optimizer.
- Increase training epochs.
- Apply dropout to reduce overfitting.

Python Code

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Sample Model
```



```
model = Sequential()

model.add(Embedding(input_dim=5000, output_dim=64))

model.add(LSTM(128))

model.add(Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Conclusion

Standard RNNs struggle with long-term dependencies due to vanishing gradients and limited memory. Replacing RNNs with LSTM or GRU, using attention mechanisms, applying gradient clipping, and improving training strategies significantly enhance NLP model performance.